IMPERIAL

MAPEM Generation Pipeline User Manual

Version: 2.0 **Issue date:** 2026-06-18

Imperial College London

MSc Transport & MSc Transport with Data Science

Group 09

Revision History

Version	Date	Description
1.0	2026-06-14	Initial draft
2.0	2026-06-18	Added Sections 7–8, generator & validation, fixes

Table of Contents

1 Overview

This manual explains how to run, understand and interpret the MAPEM generation pipeline developed in this project.

The pipeline takes a site folder containing heterogeneous traffic signal data and processes it through several stages. the input data may include CAD drawings, PDF specifications, UTC forms, controller configuration files, text reports, GIS files and other supporting files.

The overall workflow is:

```
Raw site data
    → Site inventory
    → Fact extraction
    → Geometry and semantic assignment
    → Ingestion adapter
    → MAPEM field matching
    → Evidence fusion
    → MAPEM-style generated outputs
```

The pipeline can be run in two ways:

1. Automatically, using `run_pipeline_with_generator.py`.
2. Manually, by running each stage step by step.

This automated script is mainly provided to make testing and demonstration easier; they run the existing modules in sequence but do not replace the underlying generation logic.

The generated outputs should be treated as structured evidence and draft MAPEM-style results. They still require review, especially where lane-level geometry, signal group interpretation or movement-to-lane mapping is incomplete.

2 Required Environment and Dependencies

Before running the pipeline, the required Python environment, packages and external tools should be prepared.

The project should be run from the project root, which is the folder containing:

```
pyproject.toml
README.md
src/
data/
outputs/
run_pipeline_with_generator.py
```

In the current setup, the project root is:

```
/Users/mac/PycharmProjects/MAPEM3135
```

If the project is run on another computer, replace this path with the actual project root.

2.1 Python Version

The project should be run with Python 3.11 – 3.13.

Check the Python version:

```
python --version
```

Expected output:

```
Python 3.13.
```

A dedicated Python virtual environment should be used for this project. If the project is run on Windows, please create and activate a Python 3.13 virtual environment first, then run the same `python -m pip ...` installation commands below.

Python 3.13 is recommended because some geospatial dependencies, such as Fiona, may not provide stable pre-built wheels for newer Python versions. If a newer Python version is used, `pip` may try to compile these packages locally and require additional system-level dependencies such as GDAL.

2.2 Homebrew

Homebrew can be used to install system-level tools such as Tesseract OCR.

Homebrew installation command:

```
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

Check whether Homebrew is installed:

```
brew --version
```

If the command prints a Homebrew version number, the installation is successful.

2.3 Project Virtual Environment

Go to the project root:

```
cd /Users/mac/PycharmProjects/MAPEM3135
```

Create a virtual environment:

```
python3.13 -m venv .venv
```

Activate the virtual environment:

```
source .venv/bin/activate
```

Upgrade `pip`:

```
python -m pip install --upgrade pip
```

If the virtual environment is already created, activate it before running the pipeline:

```
source .venv/bin/activate
```

For Windows users, activate the corresponding Python 3.13 virtual environment before running the installation and pipeline commands.

2.4 Project Python Dependencies

After activating the virtual environment, install the project in editable mode:

```
python -m pip install -e.
```

This installs the Python packages declared in `pyproject.toml`.

`pip install -e .` installs the five packages declared in `pyproject.toml`. Two more packages, `pyyaml` and `openpyxl`, are required at runtime but are not declared there, so they must be installed as well:

```
python -m pip install "python-docx>=1.1" "pdfplumber>=0.11" "ezdxf>=1.3" "fiona>=1.10"
"pyproj>=3.7" "pyyaml" "openpyxl"
```

The main packages are used for:

Package	Purpose
`python-docx`	Read DOCX paragraphs and tables
`pdfplumber`	Read PDF text, tables and vector drawing objects
`ezdxf`	Parse DXF and converted DWG drawings
`fiona`	Read Shapefile and GeoPackage GIS files
`pyproj`	Coordinate conversion, including British National Grid to WGS84
`pyyaml`	Read YAML files such as `matching_rules.yaml` and `site_config_<site_id>.yaml`
`openpyxl`	Read Excel files such as `MAPEM_Dictionary_v7.xlsx`

TXT, 8TX, ZIP, GeoJSON, JSON and OSM extraction mainly use the Python standard library and do not normally require additional parser packages.

2.5 OCR/CV Dependencies for Image-based PDFs

Some PDF drawings may contain embedded images rather than extractable text or vector objects. In this case, OCR and computer vision packages are needed.

Install the optional OCR/CV dependencies:

```
python -m pip install -e ".[cv]"
```

If needed, install them explicitly:

```
python -m pip install opencv-python pytesseract pymupdf
```

These packages are used for:

Package	Purpose
`opencv-python`	Detect line candidates from rendered PDF images
`pytesseract`	Python wrapper for Tesseract OCR
`pymupdf`	Render PDF pages into images for OCR/CV processing

Important: `pytesseract` requires the Tesseract OCR executable to be installed separately.

2.6 Tesseract OCR

Tesseract OCR is required when the pipeline needs to extract text from image-based or scanned PDF pages.

Official website:

```
https://tesseractocr.org
```

Check whether Tesseract is available:

```
tesseract --version
```

If the command prints a Tesseract version number, the installation is successful.

Tesseract is only required for OCR-based PDF processing. If a PDF contains extractable text or vector drawing objects, Tesseract may not be needed for that file. However, if the site contains scanned or image-based PDF drawings, missing Tesseract may cause OCR-related extraction to fail.

2.7 ODA File Converter for DWG Files

DWG files cannot be decoded directly by pure Python. The pipeline uses ODA File Converter to convert DWG files into DXF before parsing.

ODA File Converter must be installed separately.

Download link:

```
https://www.opendesign.com/guestfiles/oda_file_converter
```

For the current macOS setup, the expected executable path is:

```
/Applications/ODAFileConverter.app/Contents/MacOS/ODAFileConverter
```

Before running the pipeline, set the environment variable:

```
export ODAFC_PATH="/Applications/ODAFileConverter.app/Contents/MacOS/ODAFileConverter"
```

Check whether `ezdxf` can detect ODA File Converter:

```
python -c "from ezdxf.addons import odafc; print(odafc.is_installed())"
```

Expected output:

```
True
```

If the output is `False`, update `ODAFC_PATH` to the actual ODA File Converter executable path.

If the site folder contains `.dwg` files but ODA File Converter is not installed or not detected, the DWG extraction step may fail.

2.8 OSTN15 Grid for Higher-accuracy Coordinate Conversion

The pipeline can run without the OSTN15 grid, but coordinate conversion accuracy may be degraded.

If the following warning appears:

```
ostn15_grid_missing
```

then coordinate conversion is still running, but with lower accuracy.

To install the OSTN15 grid through `pyproj`, run:

```
python -m pyproj sync --file uk_os_OSTN15_NTv2_OSGBtoETRS.tif
```

For testing and demonstration, this warning can usually be noted rather than treated as a blocking error.

2.9 Environment Check

After completing the setup, run the following checks from the project root:

```
cd /Users/mac/PycharmProjects/MAPEM3135
```

```
source .venv/bin/activate
```

Check Python:

```
python --version
```

Expected:

```
Python 3.13.x
```

Check whether the project package can be imported:

```
python -c "import mapemgen; print('mapemgen import OK')"
```

Check YAML support:

```
python -c "import yaml; print('PyYAML OK')"
```

Check PDF support:

```
python -c "import pdfplumber; print('pdfplumber OK')"
```

Check CAD support:

```
python -c "import ezdxf; print('ezdxf OK')"
```

Check OCR Python package support:

```
python -c "import pytesseract; print('pytesseract OK')"
```

Check Tesseract OCR:

```
tesseract --version
```

Check ODA File Converter:

```
python -c "from ezdxf.addons import odafc; print(odafc.is_installed())"
```

If all required checks pass, the environment is ready for running the pipeline.

2.10 Note on MOVA Files

Some site folders may contain `.mova` files. These are proprietary MOVA dataset files and cannot be reliably decoded directly by Python.

Because the current MOVA information available to the project is limited, this manual does not include MOVA Tools installation instructions.

If MOVA information is required in the future, the recommended approach is to export readable reports or text files from official MOVA Tools first, and then place those exported files in the relevant site folder for parsing.

3 Input Data and Site Configuration File

3.1 Input Data

For a given `<site_id>`, all raw input files for that site should be placed in:

```
data/<site_id>/
```

For example, the demo site 337L uses:

```
data/337L/
```

The pipeline is designed to accept heterogeneous traffic-signal data. A site folder may contain the following file types:

File type	Typical content

PDF specification	controller specification, phase/stage tables, drawing labels
DWG / DXF drawing	junction geometry: lanes, stop lines, signal heads, layer names

DWG files cannot be read directly. They should be converted to DXF by using ODA File Converter.

Some PDF specifications are scanned images rather than extractable text. These require Tesseract OCR to extract their text.

Original file names should be kept. The site inventory step uses filename keywords (for example Spec, Drawing, UTCForm, MOVA) to decide how each file should be parsed.

3.2 Site Configuration File

In addition to the raw data, each site needs a site configuration file. This file provides per-site information that the automatic extractors cannot reliably determine on their own, such as which phases are dummy phases. For a given `<site_id>`, the configuration file must be located at:

`src/mapemgen/matching/site_config_<site_id>.yaml`

The file name must match the `<site_id>` used when running the pipeline. For example, running python `run_pipeline_with_generator.py 337L` reads:

`src/mapemgen/matching/site_config_337L.yaml`

There is one configuration file per site and the configuration file is organised into two parts.

Part A: Required for every site. These fields are read by the matching engine and must be confirmed or filled for each new site:

`site`: basic site identity, including id, name, and the input file names under `source_files`.

`site.dummy_phases`: the dummy phase letters listed in the controller specification's "USE OF PHASES" table. Dummy phases are controller placeholders, not real signal groups, and must be excluded.

`region_code`: the MAPEM region code, formed as the two-digit country code (United Kingdom = 50) followed by the three-digit local authority code. Sites under the same authority can reuse the same value.

`manual`: optional manual overrides, used only when automatic extraction is missing or wrong.

Part B: Reserve. These fields describe interfaces that are designed but not yet consumed by the current code. They can be left as provided and do not need to be filled when adding a new site.

To create a configuration for a new site, copy an existing configuration file, rename it to `site_config_<new_site_id>.yaml`, and update only Part A. Part B can be left unchanged.

4 Running the Pipeline

4.1 Automated Execution Using run_pipeline_with_generator.py

After the environment has been configured and the site data prepared, the full pipeline can be run using the automated wrapper:

`run_pipeline_with_generator.py`

This script runs all stages in sequence — through to the final MAPEM output — so the user does not need to enter each command manually.

Run it from the project root. To run the demo site 337L:

```
cd <project-root>

python run_pipeline_with_generator.py 337L
```

Here 337L is the site ID. The general format is python run_pipeline_with_generator.py <site_id>; to run another site, replace it, e.g. python run_pipeline_with_generator.py 1003.

For a given `<site_id>`, the script expects input data in `data/<site_id>/` and the configuration at `src/mapemgen/matching/site_config_<site_id>.yaml`. Outputs are written to `outputs/<site_id>/`.

On success the Terminal prints Pipeline finished. and lists the main outputs:

`site_inventory.partial.json`

`extracted_facts.partial.json`

`geometry_assignments.partial.json`

`facts.<site_id>.json`

`mapped_evidence.<site_id>.json`

`fused_model.<site_id>.json`

`fusion_report.<site_id>.json`

`mapem.<site_id>.json`

`mapem.<site_id>.asn1`

If no site ID is given, it defaults to 337L; including it explicitly is recommended.

This is an automation wrapper that runs the existing modules in sequence; the actual generation logic lives in the modules under `src/mapemgen/`.

4.2 Optional Manual Step-by-step Execution

The same workflow can also be run manually. This is useful if a specific stage needs to be checked or debugged. The example below uses the demo site `337L`.

Step 1: Site Inventory

From the project root:

```
python -m mapemgen.cli inventory \
  --site-folder data/337L \
  --site-id 337L \
  --out-dir outputs/337L
```

Output:

```
outputs/337L/site_inventory.partial.json
```

Step 2: Fact Extraction

```
python -m mapemgen.cli extract \
  --site-folder data/337L \
  --site-id 337L \
  --out-dir outputs/337L
```

Output:

```
outputs/337L/extracted_facts.partial.json
```

Step 3: Geometry and Semantic Assignment

```
python -m mapemgen.cli assign-geometry \
  --input outputs/337L/extracted_facts.partial.json \
  --out-dir outputs/337L
```

Output:

```
outputs/337L/geometry_assignments.partial.json
```

Step 4: Ingestion Adapter

Move to the matching directory:

```
cd src/mapemgen/matching
```

Run:

```
python ingest_adapter.py \
  --facts ../../../outputs/337L/extracted_facts.partial.json \
  --assignments ../../../outputs/337L/geometry_assignments.partial.json \
  --out ../../../outputs/337L/facts.337L.json
```

Output:

```
outputs/337L/facts.337L.json
```

Step 5: MAPEM Field Matching

From the matching directory:

```
python matching_engine.py \
  --rules matching_rules.yaml \
  --facts ../../../outputs/337L/facts.337L.json \
  --config site_config_337L.yaml \
  --transforms transforms_overlay \
  --out ../../../outputs/337L/mapped_evidence.337L.json
```

Output:

`outputs/337L/mapped_evidence.337L.json`

Important:

`--transforms transforms_overlay`

should be included. Otherwise, many fields may become `pending_transform`.

Step 6: Evidence Fusion

Return to the project root:

```
cd /Users/mac/PycharmProjects/MAPEM3135
```

Run:

```
python src/mapemgen/fusion/fuse.py \
  --evidence outputs/337L/mapped_evidence.337L.json \
  --out outputs/337L/fused_model.337L.json \
  --report outputs/337L/fusion_report.337L.json
```

Outputs:

`outputs/337L/fused_model.337L.json`

`outputs/337L/fusion_report.337L.json`

Step 7: Generation

Run:

```
python src/mapemgen/generators/run_generator.py \
  --input outputs/337L/fused_model.337L.json \
  --out-dir outputs/337L
```

Outputs:

`outputs/337L/mapem.json`

`outputs/337L/mapem.asn1`

Step 8: Validation

Run:

```
python -m mapemgen.cli validate \
  --input outputs/337L/fused_model.337L.json
```

Outputs:

```
The validation report is printed to the terminal (it is not written to a file).
The command exits with code 0 if the model is usable, or 2 if it is not.
```

5 Generation Workflow and Key Code Files

This chapter explains the role of each stage in the MAPEM generation workflow, the key code files, the expected inputs and outputs.

```
site folder
  -> site_inventory.partial.json
  -> extracted_facts.partial.json
  -> geometry_assignments.partial.json
  -> facts.<site_id>.json
  -> mapped_evidence.<site_id>.json
  -> fused_model.<site_id>.json + fusion_report.<site_id>.json
  -> mapem.<site_id>.json + mapem.<site_id>.asn1
```

5.1 Site Inventory

This stage is a file-level EDA check only. It records file paths, file types, file sizes, readability status, filename hints and the recommended parser for each source file. It does not parse PDF tables, read CAD geometry or produce MAPEM fields. Users should inspect `inventory_summary` and `source_files[]` to confirm that key files have been found, such as configuration PDFs, drawing PDFs, UTC forms, RAM/8TX files, DWG/DXF drawings, MOVA files or GIS data.

Main code file: `src/mapemgen/ingestion/inventory.py`

CLI entry: `src/mapemgen/cli.py`

Main output: site_inventory.partial.json

```
python -m mapemgen.cli inventory
  --site-folder "<site-folder>"
  --site-id "<site-id>"
  --out-dir "outputs/<site-id>"
```

5.2 Fact Extraction

The extraction stage recursively scans the site folder and dispatches each file to the appropriate parser. Each extracted fact should retain its `fact_name, payload, confidence,`

evidence_location and source_file. The confidence value is an engineering confidence level, not a statistically calibrated probability, and it does not mean that the fact has already been accepted as a final MAPEM field. DWG files require ODA File Converter. PDF image recognition requires optional OCR/CV dependencies. MOVA binary files are not reverse-engineered directly; full MOVA evidence should come from exports produced by the official MOVA Tools application.

Main coordinator: src/mapemgen/ingestion/facts.py

Important parser files: cad.py, pdf_tables.py, pdf_cv.py, docx_tables.py, ram_text.py, gis.py, mova.py, zip_packages.py, movement_tables.py

Main output: extracted_facts.partial.json

```
python -m mapemgen.cli extract
  --site-folder "<site-folder>"
  --site-id "<site-id>"
  --out-dir "outputs/<site-id>"
```

5.3 Geometry and Semantic Assignment

This stage does not decide final MAPEM fields and does not rewrite the original parser facts. It adds scope information such as target_scope.intersection_ref, target_scope.approach_ref, target_scope.lane_ref, phase_ref, stage_ref, detector_ref or signal_group_ref. The code prioritises stronger lane sources, usually CAD first, then GIS or PDF fallback. For heuristic CAD lanes, the program checks contextual evidence such as stop lines, signal heads, arrows, movement labels and road markings. If the evidence is insufficient, the lane is kept as requires_context_match rather than being treated as a confirmed MAPEM lane.

Main code file: src/mapemgen/assignment/geometry.py

Main output: geometry_assignments.partial.json

```
python -m mapemgen.cli assign-geometry
  --input "outputs/<site-id>/extracted_facts.partial.json"
  --out-dir "outputs/<site-id>"
```

5.4 Ingestion Adapter

The adapter is the boundary between upstream parsing and the matching engine. It flattens source_files[].extracted_facts, normalises payloads, converts numeric confidence into high, medium or low, and injects intersection_ref and lane_ref from the geometry-assignment stage into each fact payload. For phase or movement facts, the adapter uses movement_lane_mappings[] to route movements or phases to lanes. If the mapping still requires contextual confirmation, the output is marked as provisional rather than silently confirmed.

Main code file: src/mapemgen/matching/ingest_adapter.py

Main output: `facts.<site_id>.json`

```
python src/mapemgen/matching/ingest_adapter.py
  --facts "outputs/<site-id>/extracted_facts.partial.json"
  --assignments "outputs/<site-id>/geometry_assignments.partial.json"
  --out "outputs/<site-id>/facts.<site-id>.json"
```

5.5 MAPEM Field Matching

The matching stage is data-driven. `MAPEM_Dictionary_v7.xlsx` and `generator_overlay.json` generate `matching_rules.yaml`, and users should generally avoid hand-editing the generated field definitions. The engine expands MAPEM paths containing [], gathers candidate evidence by fact group, and selects evidence using a final score. The output preserves `source_facts`, `transforms_run`, `priority_used`, conflict, corroborating and status. If a required group is missing, a transform is not yet implemented or the payload shape does not match the contract, the field becomes `manual_review_required`, `pending_transform` or `unresolved`. The run should degrade with traceable output rather than fail without explanation.

Main files: `src/mapemgen/matching/matching_engine.py, matching_rules.yaml, transform_pipelines.yaml, transforms.py, transforms_overlay.py, site_config_<site_id>.yaml`

Main output: `mapped_evidence.<site_id>.json`

```
python src/mapemgen/matching/matching_engine.py
  --rules "src/mapemgen/matching/matching_rules.yaml"
  --facts "outputs/<site-id>/facts.<site-id>.json"
  --config "src/mapemgen/matching/site_config_<site-id>.yaml"
  --transforms transforms_overlay
  --out "outputs/<site-id>/mapped_evidence.<site-id>.json"
```

5.6 Evidence Fusion

Fusion reads `mapped_evidence`, parses each concrete `target_path`, and assembles the nested `header, mapData, intersections, laneSet, connectsTo` and `related structures`. Fields with matched are accepted. Fields with `matched_with_conflict` are accepted but recorded under conflicts. `manual_review_required` fields with a value are retained as provisional values. `unresolved` or `pending_transform` fields are written as null and recorded as gaps. The `fusion_report` is an essential review file because it explains which values were accepted, which require human confirmation, which fields remain empty, and whether generic consistency checks were triggered. Fusion does not perform ASN.1 encoding.

Main code file: `src/mapemgen/fusion/fuse.py`

Main outputs: `fused_model.<site_id>.json, fusion_report.<site_id>.json`

```
python src/mapemgen/fusion/fuse.py
```

```
  --evidence "outputs/<site-id>/mapped_evidence.<site-id>.json"
  --out "outputs/<site-id>/fused_model.<site-id>.json"
  --report "outputs/<site-id>/fusion_report.<site-id>.json"
```

5.7 MAPEM Output Generation

This is the final stage. It reads the fused model and produces the MAPEM-style outputs: a human-readable `mapem.json` and an `ASN.1` value-notation representation `mapem.asn1`. This stage runs automatically as the final step of run_pipeline_with_generator.py. It can also be run on its own (see Section 4, Step 7 of the manual execution).

The JSON generator normalises the fused model into MAPEM-facing fields, removes internal lane-level maneuvers, and encodes the relevant bitstring fields. The `ASN.1` generator builds on this JSON and formats it into `ASN.1` value notation.

Because this stage builds directly on the fused model, any empty or low-confidence fields from fusion are carried through. For example, if refPoint or the intersection id was not resolved, the generated MAPEM will still be incomplete.

5.8 Validation Report

This stage checks the assembled model and reports whether it is usable as a MAPEM result. It does not change the model; it summarises the model's completeness and structural health.

The validation report records the `site_id`, an `is_usable flag, a list of errors, a list of warnings, and metrics such as the intersection count, lane count, lane-node count, connection count, referenced signal-group count and signal-head-location count`. A model with unresolved mandatory fields or structural problems is reported as not usable.

There are two entry points:

- `python -m mapemgen.cli validate prints the report and returns a non-zero exit code when the model is not usable.`
- `python -m mapemgen.cli generate writes mapem.json, mapem.asn1 and validation_report.json together.`

Main code:
files: `src/mapemgen/validation/report.py, src/mapemgen/pipeline.py, src/mapemgen/cli.py`
Main output:

`validation_report.json`

```
python -m mapemgen.cli validate \
  --input "outputs/<site-id>/fused_model.<site-id>.json"
```

6 Key Terms and Status Meanings

This section defines the key terms, status values and warnings used throughout the MAPEM generation pipeline. It explains how evidence changes as it moves from fact extraction through

assignment, matching, and fusion. Use the tables below to determine what each term means, how it affects the final MAPEM output, and what should be checked before a result is accepted.

6.1 Cross-stage Concepts

Terms / Status	Meaning	Interpretation
candidate fact	An observation or inferred item retained as possible evidence for later stages.	It has not yet been selected for a MAPEM field. Even a high-confidence candidate can be rejected, scoped differently, or left unresolved during matching and fusion.
final MAPEM value	The value placed at a concrete MAPEM target path after matching and fusion decisions.	A final value may be accepted normally, accepted with a recorded conflict, or retained provisionally. Its presence does not by itself prove that every source agreed.
`source_file`	The source file from which a fact originated.	It provides file-level provenance, but may not identify the precise page, table row, archive member, or CAD entity.
warning	A non-fatal condition indicating degraded accuracy, incomplete coverage, or an item requiring attention.	The pipeline may still complete and produce usable output. The impact depends on the warning, for example reduced coordinate accuracy versus unknown fact names.
error	A failure or inconsistency severe enough to stop an operation or make part of the result structurally invalid.	An extraction dependency error may stop a stage, while a fusion consistency issue with severity: error may allow file creation but still block approval or encoding.
null	An explicit absence of a resolved value.	It is not numeric zero, an empty string, or proof that the real-world attribute does not exist. Fusion commonly writes null for unresolved or pending fields.
`ostn15_grid_missing`	A warning that the high-accuracy OSTN15 coordinate grid is unavailable.	BNG-to-WGS84 conversion can still run, but expected accuracy is degraded to approximately metre-level. It is not a pipeline failure.

6.2 Site Inventory

Terms / Status	Meaning	Interpretation
`inventory_summary`	A file-level summary of what the site inventory discovered.	It supports source-data inspection only. It does not prove that PDF tables, CAD geometry, or MAPEM facts were successfully extracted.

`file_type`	The detected or inferred source-file format.	It determines which parser can be recommended or dispatched. A misleading extension may lead to an unsuitable parser or a parse failure.
`readability status`	An inventory indication of whether the file can be accessed at the filesystem level.	Readable does not mean semantically parseable. A readable binary or damaged document may still fail during extraction.

6.3 Fact Extraction

Term / Status	Meaning	Interpretation	What to Check
parsed	The relevant parser completed for the file without a recorded file-level parse failure.	It does not guarantee that useful facts were found, every page or entity was understood, or extracted candidates are correct.	--
unsupported	The file was detected, but its format is not supported by the current extraction dispatcher.	No content facts are produced. This differs from parser_error, where a supported parser was attempted and failed.	Decide whether the file contains required evidence and whether it should be converted, exported, or supported by a new parser.
skipped	The file was intentionally not parsed because a defined filtering or deduplication rule applied.	● Skipping can be correct, for example for non-site CAD sources or duplicate CAD inside a ZIP. ● Not automatically an extraction failure.	--
`parser_error`	A supported parser failed for one source file because of corruption, unexpected structure, or another file-level exception.	● The coordinator normally records the error and continues with other files. ● Evidence from the failed file is absent and may reduce downstream coverage.	Inspect the recorded message, open the file manually, verify dependencies, and rerun extraction after correction.
`extracted_facts`	The candidate facts produced by a parser for a source file.	The list may contain direct metadata, structured rows, raw geometry, or weak semantic inferences. Different items can require different review standards.	--

`fact_name`	The canonical fact identifier used by matching rules to find relevant evidence.	A correctly extracted value may still be ignored if its fact name is unknown, misspelled, or not associated with the required fact group.	• Compare unknown-name warnings with the MAPEM dictionary and extraction contract • Correct the pipeline producer rather than renaming output manually.

6.4 Geometry and Semantic Assignment

Term / Status	Meaning	Interpretation	What to Check
`assigned_facts[]`	Geometry facts enriched with an intersection scope and, where defensible, a lane scope.	Assignment adds scope without deleting or rewriting the original parser fact.	Compare the assigned scope, method, and distance with the source geometry and coordinate space.
`semantic_assignments[]`	Non-geometric facts enriched with direct semantic references such as phase, stage, detector, or approach.	A semantic reference can be valid at intersection level while `lane_ref` remains null.	--
`movement_lane_mappings[]`	Conservative mappings from a structured movement reference to a lane reference.	Mappings are strongest when the lane source exposes a matching movement label. Proxy-based mappings remain provisional.	Review `assignment_method`, `requires_context_match`, phase references, and the source label or geometry supporting each mapping.

`lane_ref`	A stable internal reference for an assigned or proxy lane object.	The existence of a lane reference does not by itself distinguish a geometry-derived lane from a directional or semantic proxy.	--
`lane_ref: null`	The fact has no defensible lane-level assignment.	The fact can still be valid at intersection, approach, phase, stage, or detector scope. It should not be forced onto the nearest lane without a reliable anchor.	Look for missing movement labels, incompatible coordinate space, or insufficient geometry context before adding a manual mapping.
`assignment_method`	The named method explaining how a scope or movement-to-lane relationship was produced.	It is essential for distinguishing direct semantic extraction, spatial proximity, labelled geometry, and fallback proxies.	● Use the method with source evidence and context flags ● Not judge reliability from `lane_ref` alone.
`requires_context_match: true`	A flag marking a candidate relationship that still requires spatial, semantic, or human confirmation.	The relationship may be retained for routing or testing, but it is not a confirmed lane mapping.	● Seek independent labels, geometry, structured tables, or manual review ● Not directly clear the flag.
`unmatched_reason`	A diagnostic reason explaining why a movement or fact could not be assigned to a lane.	For example, `no_lane_movement_label` indicates missing contextual evidence rather than a parser crash.	Use the reason to decide whether to improve extraction, assignment rules, site configuration, or manual mapping.

`unassigned`	A fact that could not be converted to a usable point, bounds, centroid, or semantic scope.	The original fact remains available, but it cannot safely participate in scoped lane-level matching.	Check payload geometry, coordinate fields, source metadata, and whether the fact is inherently non-geometric.

6.5 Field Matching

Term / Status	Meaning	Interpretation	What to Check
`matched`	A usable value was selected for the concrete MAPEM field without a recorded source conflict requiring status escalation.	Fusion normally accepts and writes the value.	--
`matched_with_conflict`	A value was selected, but one or more eligible sources disagree with the chosen value.	● Not a failed match. ● Fusion normally notes the selected value and records the disagreement under conflicts.	Review conflict, source priority, scoring, tolerance, value differences, and source independence before approval.
`manual_review_required`	The engine cannot safely confirm the field without human judgement.	● Common causes include missing required fact groups, provisional movement-to-lane mappings, uncertain coordinate declarations, or evidence that needs domain confirmation. ● A value may or may not be present.	Check conflict.missing_groups, notes, source facts, and relevant site configuration.

`unresolved`	Candidate evidence was available, but matching could not derive a usable field value.	● The issue can be insufficient evidence, incompatible candidate shapes, failed selection, or no safe fallback. ● Fusion writes null and records a gap.	Inspect candidate source facts, missing groups, payload shape, rule coverage, and whether assignments were supplied.
`pending_transform`	Source evidence matched the field, but a required transform was missing, failed, or could not consume the payload shape.	● This usually indicates an implementation or data-contract problem rather than complete absence of source evidence. ● Fusion writes null and records a gap.	Confirm transforms_overlay was used, then inspect transforms_run, notes, function availability, and payload keys/types.
`conflict`	Structured details describing disagreement, missing requirements, or another reason the field could not be treated as an uncomplicated match.	● The exact contents depend on status. ● Under matched_with_conflict, it explains competing values ● Under review states, it may identify missing groups.	Compare all competing values, chosen evidence, tolerance, scoring, and missing requirements.

6.6 Evidence Fusion

Term / Status	Meaning	Interpretation	What to Check
`accepted`	A field value included in the fused model as the selected usable result.	● Accepted values normally come from matched fields ● conflict-bearing matches may also be included while separately reported.	--

`provisional`	A value retained in the fused model while still requiring manual confirmation.	It commonly comes from `manual_review_required` with a usable value.	Review the linked reason, missing groups, source evidence, scope, and site configuration before final approval.
`gaps`	Fields that remain empty after fusion.	• They normally correspond to unresolved, `pending_transform`, or review-required fields without a usable value. • The model records null at those paths.	Trace each gap to the matching status and decide whether to improve extraction, assignment, rules, transforms, or manual configuration.
`needs_review`	The report list of retained values or decisions requiring manual confirmation.	These items may already appear in the fused model as provisional values.	Resolve or explicitly approve every item before treating the model as final.
`conflicts`	The report list of fields whose selected value was accompanied by disagreement between sources.	• The selected value can remain in the model • The list preserves the unresolved disagreement for review.	Compare values, priorities, and provenance and decide whether the selected value is defensible or fusion policy needs adjustment.
`consistency_issues`	Structural or logical validation findings across the assembled model.	They can reveal references to nonexistent lanes, duplicate IDs, invalid nesting, or other relationships that per-field matching cannot detect.	• Review all entries and prioritise those with severity: error • Trace structural issues to fusion or upstream lane generation.
`severity: warning`	A consistency finding that warrants attention but does not necessarily make the model unusable.	The model can still be reviewed or tested, depending on the affected relationship and intended use.	Document the decision to accept or correct it and confirm it does not affect encoding-critical structure.

`severity: error`	A consistency finding indicating a materially invalid or self-contradictory model relationship.	The pipeline may still write files, but the result should not be approved for encoding or delivery until resolved.	Fix the fusion rule or upstream structure, rerun the affected stages, and confirm the error no longer appears.

7 Known Limitations

7.1 Georeferencing / refPoint

The coordinates in CAD drawings are real British National Grid values, but these drawing files usually don't include a machine-readable coordinate reference system. However, the pipeline requires an explicit coordinate reference to evaluate refPoint. If this reference is missing, refPoint can be null. Considering that refPoint is mandatory under C-Roads rules and all the geometry grids are based on refPoint, we must get the refPoint data, otherwise it directly affects the output. So the site configuration file should provide a confirmed coordinate reference, or the refPoint can be given in the manual review stage.

7.2 CAD Geometry Filtering

The CAD extraction is not able to differentiate real road geometry from the drawing's auxiliary elements like the legend, key table and title block. When the drawings lack dedicated lane-centreline layer, the clustering heuristic may mistake these auxiliary lines for lanes, leading to a wrong number or position of lanes.

7.3 Lane Type Semantics

When there is no recognisable sources for laneType, it falls back to the default "vehicle". This is a real problem because types like pedestrian crossing and bicycle path may not be identified. So it may need a manual review.

7.4 Geometry Transforms Depend on refPoint

Also, fields like the ingress and egress approach, and the signal head nodeXY are evaluated based on refPoint. When refPoint can't be resolved, these transforms can't produce any result, so the fields will be marked with `pending_transform`.

7.5 Official Intersection id

Intersection id needs a unique official intersection identifier and it is a mandatory field. It should be provided and confirmed by the client or an official source.

7.6 Input-Format Limitations

Scanned PDF: some specifications are scanned images rather than extractable text, so they rely on OCR, which leads to low confidence.

MOVA file: it exists as proprietary binary, so for now it can only be recorded, not parsed.

OSTN15 grid: if OSTN15 grid is not installed, the precision of coordinate conversion from BNG to WGS84 may decrease.

8 Post-generation Checks

8.1 Where to Look

We suggest doing a few checks, in order, before you use a generated MAPEM. These checks focus on `fusion_report.<site_id>.json` and `fused_model.<site_id>.json` in `outputs/<site_id>/`.

8.2 Read the Summary First

Open the fusion report and read the summary, it gives the overall picture: `total_leaf_fields`, `accepted`, `gaps`, `provisional_needs_review`, `conflicts`, `consistency_issues`, and `consistency_errors`. Based on this summary, we can judge whether the result is usable: if accepted is low or there are many gaps, it still needs a lot of manual completion.

8.3 Check Mandatory Fields

You should make sure the two mandatory C-Roads fields are filled. For example, check whether `refPoint` (lat / long) is null and whether the intersection id is null. If either of them is null, this output will not be a valid MAPEM, and you need to fill it in during the review stage.

8.4 Sanity-Check Geometry

Even if the refPoint is not null, you should check that it is in the right position. Divide lat / long by 10^7 to get degrees, then compare it on a map to see whether it falls on the real location of the junction.

In the fused model, check that laneSet has a reasonable number of lanes for the junction, and that each lane's coordinates fall near the real junction rather than clustering in a drawing corner such as the legend.

8.5 Review Flagged Items

Each field's status should also be checked.

- `manual_review_required`: needs manual confirmation or completion.
- `pending_transform`: the transform did not produce a result, usually related to `refPoint` or geometry.
- `unresolved`: could not be built (for example, the lane connection `connectsTo`).

For empty or wrong fields, supply or override them in the site configuration file, or correct them during manual review, then re-run the pipeline and repeat the checks above.